

Performance Analysis of Branch Predictors: A Simulation-Based Study with gem5

Sai Kaushik S
2025CSZ8470

Indian Institute of Technology Delhi

November 26, 2025

1 Introduction

The complete source code, simulation scripts, and data used for generating the plots in this report are publicly available on GitHub at

<https://github.com/sai-kaushik-s/gem5-branch-predictors>.

2 Experimental Setup

The host system specifications are as follows:

- **Processor:** AMD Ryzen 7 5800H (8 Cores, 16 Threads, 3.2GHz Base Frequency, 4.4GHz Max Frequency, 4MB L2 Cache, 16MB L3 Cache)
- **Memory:** 16 GB DDR4-3200 MHz (Running in Dual Channel)
- **Instruction Set Architecture:** x86 64-bit
- **Operating System:** Pop!_OS 22.04 LTS

The analysis was conducted using the **gem5 simulator (v25.0)** [1] [2] [3] in Full-System mode. In full system mode, gem5 acts more like an emulator (e.g., QEMU) or hypervisor (e.g., VMware ESX and Xen) than a traditional simulator. In full system mode, gem5 simulates all of the hardware from the CPU to the I/O devices. This allows gem5 to execute binaries with no modifications.

The simulation system specifications are as follows:

- **Processor:** SimpleSwitchableProcessor (1 Core, 3.2GHz Base Frequency, 128KB Shared L2 Cache, 16KB Instruction and Data Private L1 Cache)
- **Memory:** 3 GB SimpleMemory (Running in Single Channel)
- **Instruction Set Architecture:** x86
- **Kernel:** x86-linux-kernel-5.4.49
- **Disk Image:** x86-parsec

The processor configurations are as follows:

- **Starting Processor:** KVM (Host processor)
- **Switch Processor:** O3
- **Number of ROB Entries:** 256
- **Number of IQ Entries:** 128
- **Fetch, Decode, Rename, Issue, WriteBack, Commit Width:** 4

3 Branch Predictors

Below are the branch predictors experimented with:

3.1 Bimodal

Bimodal is the simplest form of branch predictors. It uses a 1-bit or 2-bit state for each branch instruction in the program, which is indexed by part of the instruction's address. Branch Prediction Table (BPT) is a table of 1-bit or 2-bit entries indexed by the lower bits of the program counter (PC). When a branch is encountered, the PC is hashed or directly indexed into the BPT. The corresponding entry is checked to predict whether the branch will be taken or not.

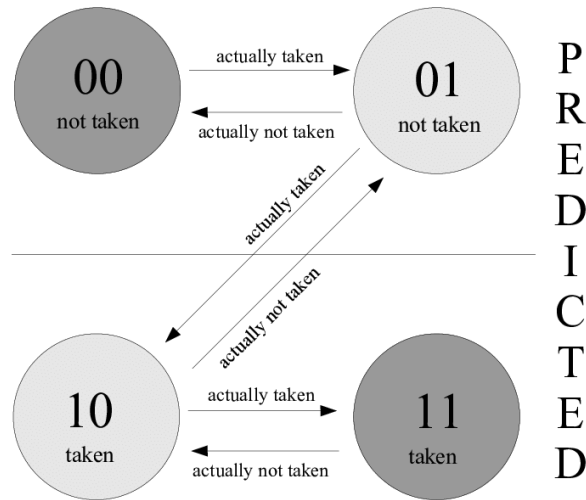


Figure 1: Bimodal Branch Prediction State Transition

3.2 Local

Local branch prediction focuses on the behaviour of individual branches. It uses a local history register (LHR) for each branch to store its past outcomes and leverages this local context for prediction.

The LHR for a specific branch is used as an index into a Pattern History Table (PHT). Each entry in the PHT contains a 2-bit saturating counter that predicts whether the branch will be Taken or Not Taken.

3.3 Tournament

A tournament predictor is a hybrid branch predictor that pits two or more different predictors against each other and dynamically chooses the one that is performing better for a given branch. It combines the strengths of different prediction strategies to achieve higher overall accuracy. The most common configuration uses a local predictor and a global predictor.

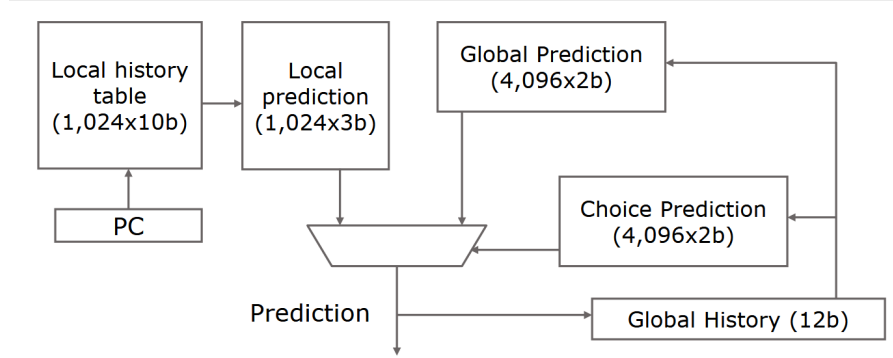


Figure 2: Tournament Branch Predictor Block Diagram

3.4 TAGE

The TAGE (TAgged GEometric history length) predictor is a sophisticated global branch predictor that uses multiple prediction tables, each indexed with a different and geometrically increasing length of global branch history. By using tags to reduce conflicts and prioritizing predictions from tables that use longer history.

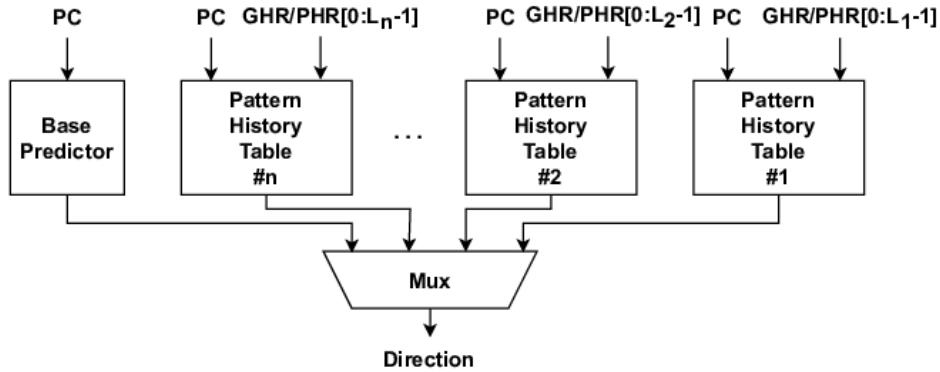


Figure 3: TAGE Branch Predictor Block Diagram

3.5 L-TAGE

The L-TAGE predictor is a hybrid design that enhances the powerful TAGE predictor by adding a specialized component to handle a specific scenario where TAGE can be inefficient: simple, long-running loops.

Essentially, an LTAGE predictor is a standard TAGE predictor combined with a Loop Predictor.

3.6 Perceptron

The Perceptron predictor is a sophisticated branch predictor that uses a simple neural network element, a perceptron, to make predictions. It learns to recognize complex patterns in the global history of branches by assigning different weights or importance levels to each past branch outcome.

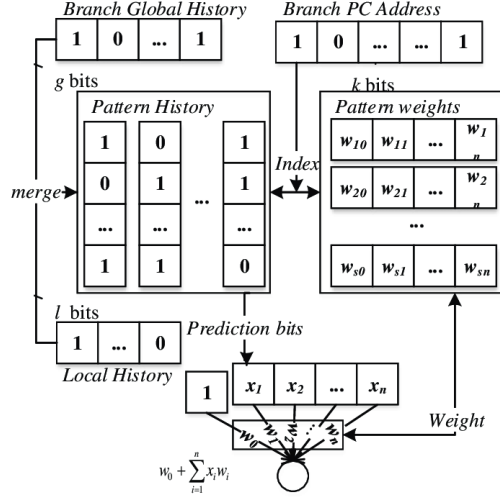


Figure 4: Perceptron Branch Predictor Block Diagram

4 Stats and Metrics

Below are some of the stats collected from the *stats.txt* file from some of the benchmark runs.

Table 1: Performance, Penalty, and Runtime for the **canneal-large** Benchmark

Predictor	IPC	Mispredict Rate (%)	Squashed Insts (M)	Sim. Seconds (s)
Bimodal	1.996	5.67	596.5	1.897
Local	2.007	4.22	480.3	1.889
Tournament	2.040	4.20	481.4	1.870
Tage	2.009	4.21	479.8	1.883
LTAGE	1.983	5.66	595.5	1.906
Perceptron	2.018	4.22	480.7	1.879

The benchmark, canneal, involves simulated annealing, which typically has complex and less repetitive control flow, making it a good stress test for branch predictors.

The more advanced predictors like TAGE, Perceptron, and Tournament perform significantly better than the others. They all achieve a much lower Mispredict Rate (around 4.2%). This directly leads to fewer Squashed Instructions (around 480M).

Bimodal is a simple predictor and is easily confused by the complex branch patterns in canneal. It's unusual for LTAGE to perform this poorly, as it's an advanced predictor. This result suggests that for this specific benchmark, its internal tables might be experiencing significant aliasing.

The blackscholes benchmark is used in financial modeling and often contains tight, predictable loops. This makes its branches generally easy to predict.

Table 2: Performance, Penalty, and Runtime for the `blackscholes-small` Benchmark

Predictor	IPC	Mispredict Rate (%)	Squashed Insts (M)	Sim. Seconds (s)
Bimodal	89.247	2.69	13.7	0.017
Local	89.775	2.70	13.7	0.017
Tournament	89.617	2.69	13.7	0.017
Tage	136.934	2.71	13.7	0.017
LTAGE	89.781	2.70	13.7	0.017
Perceptron	89.620	2.72	13.8	0.017

Mispredict Rate is very low (around 2.7%) and nearly identical for all predictors. The number of Squashed Instructions is also the same across the board. This tells us that the branches in `blackscholes-small` are so simple that even a basic Bimodal predictor can guess their direction correctly most of the time.

The advanced features of Tournament, TAGE, and Perceptron offer no real advantage here because there aren't complex patterns to learn.

The IPC of 136.9 for TAGE is a massive outlier. Given that its misprediction rate is the same as the others, this is most likely a data error or a simulation artifact.

5 Analysis

In practical terms as seen in Fig. 5, the gap from TAGE to perceptron (18%) and to tournament (60%) quantifies the performance sacrificed when a predictor cannot capture as many branch-history patterns.

TAGE's multiple, variably-tagged history tables let it match both short and very long branch correlations while avoiding destructive aliasing, giving it a systematic accuracy edge.

A perceptron learns long global histories with a compact set of weights, but its linear model misses non-linear patterns that TAGE's table voting can catch.

Local history outperforms bimodal because keeping a short per-branch history remembers idiosyncratic loop behavior, whereas a single global counter cannot.

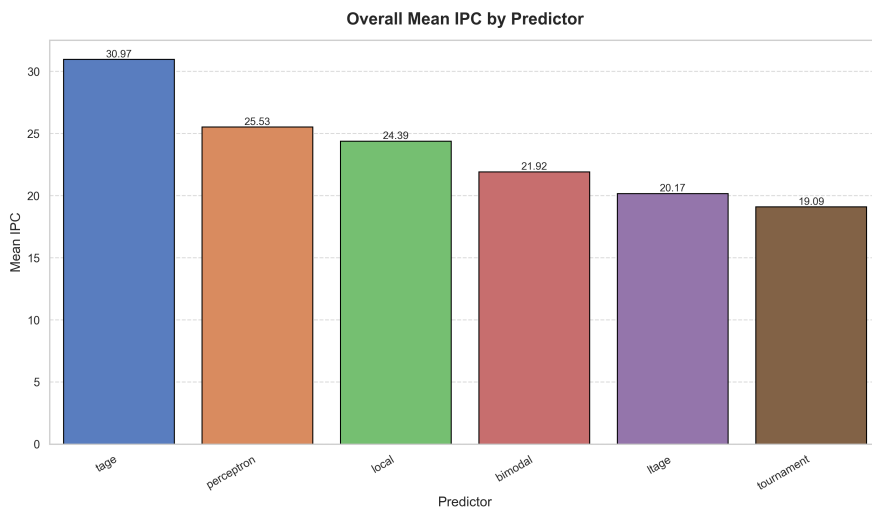


Figure 5: Overall Mean IPC vs Branch Predictors

The pair plot as in Fig. 6 shows every pair-wise relationship between five micro-architectural metrics, colour-coded by branch-prediction scheme.

Across the diagonal, the blue TAGE curve sits furthest right for IPC and furthest left for branch MPKI, confirming its high performance and low miss rate; perceptron (orange) is the next best, with bimodal/tournament (purple/brown) showing the heaviest tails toward low IPC and high misprediction density.

The ROB full events and IQ full events panels show modest spread among predictors: nearly identical kernel shapes imply that reorder-buffer and issue-queue pressure vary more with workload than with predictor choice.

- IPC vs. MPKI displays the expected inverse trend: higher misprediction density drags ipc down.
- Squashed Instruction vs. MPKI is almost linear: lower MPKI have fewer late-stage squashes.
- IPC against ROB full events or IQ full events shows little correlation: the branch accuracy, not queue pressure, is the primary differentiator between predictors.

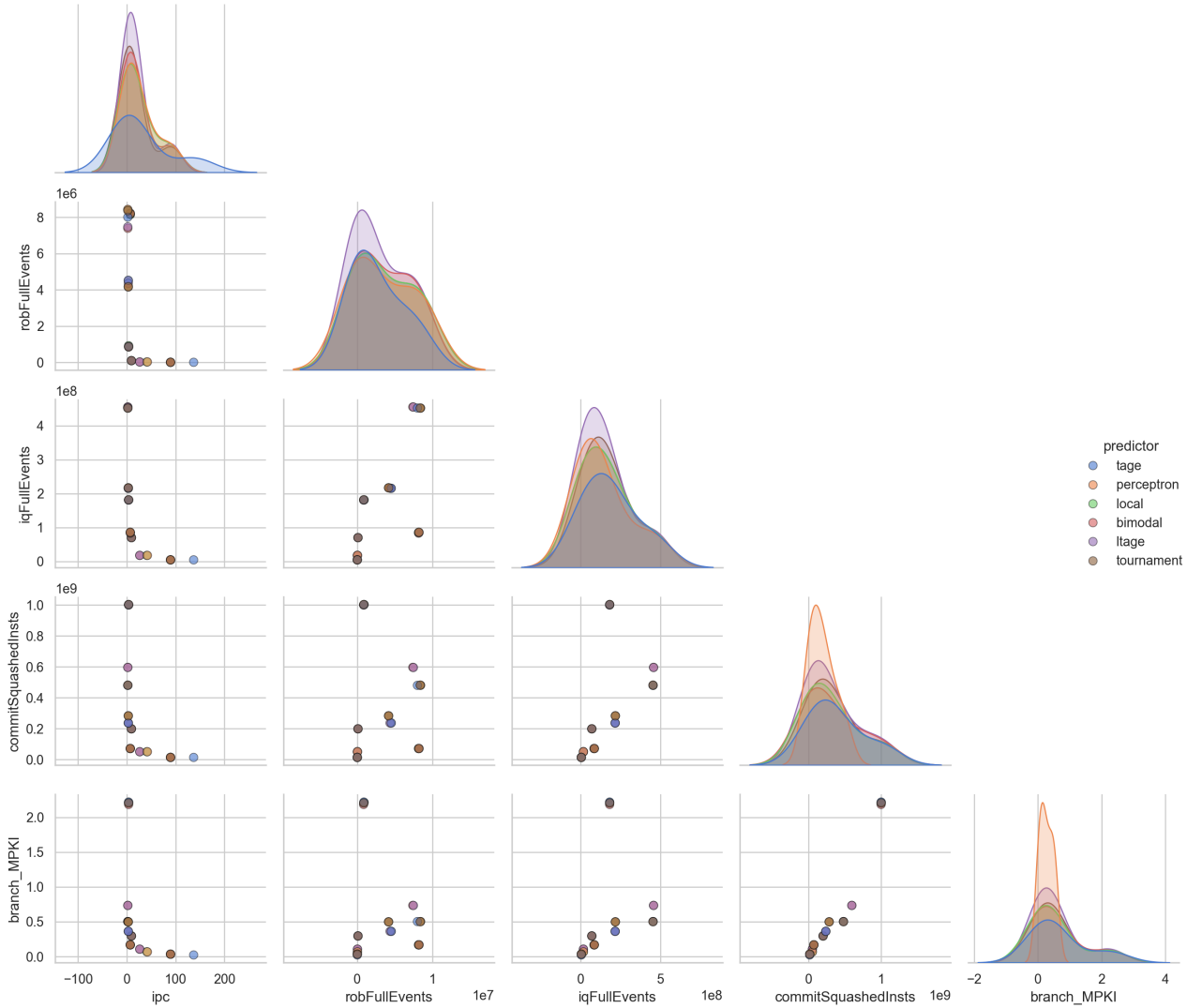


Figure 6: Pair Plot of IPC, ROB and IQ Events, MPKI

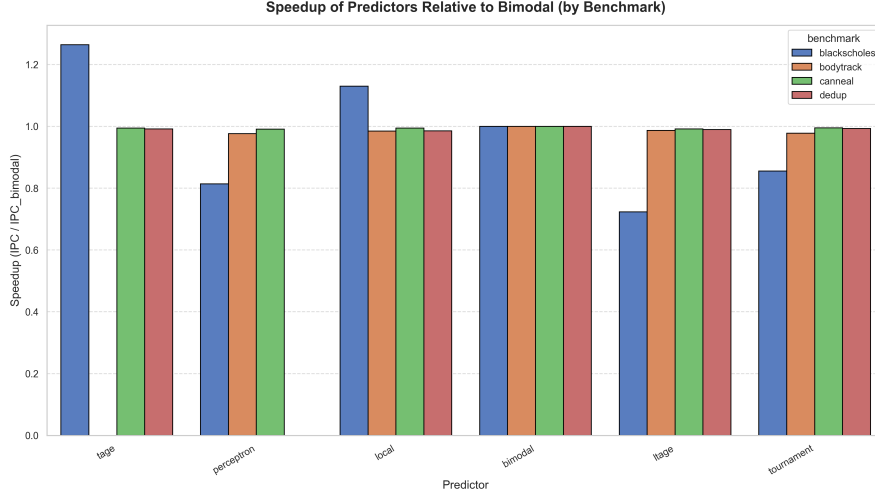


Figure 7: Speedup of the predictors with Bimodal as a baseline

Consider the Bimodal Branch Predictor as a baseline and comparing it with the other predictors as shown in Fig. 7, we can get the following inferences:

- TAGE gives the only clear win across every benchmark leveraging the multi-length tables.
- Perceptron is highly workload-sensitive as its global weight vector becomes “diluted” from non-linear patterns.
- Local predictor outperforms bimodal on every workload getting an edge from remembering each branch’s own short history.
- The trimmed tables in LTAGE and the rigid two-level structure in Tournament fails over bimodal for these datasets.

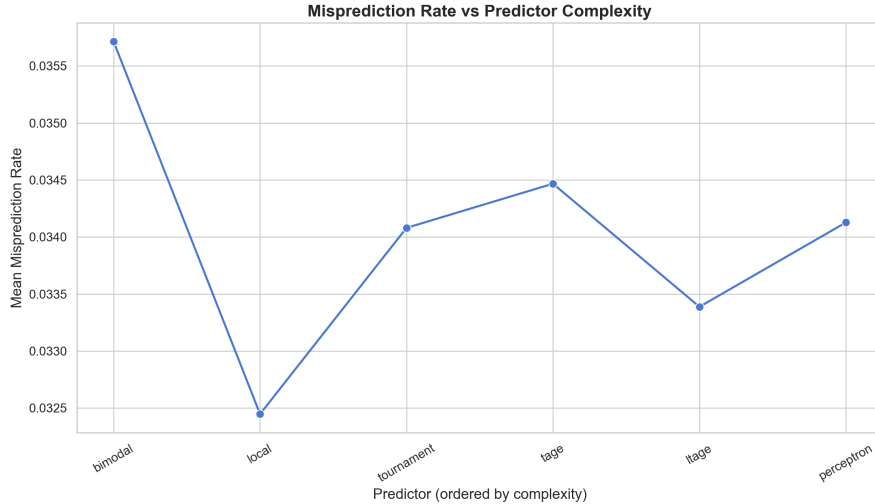


Figure 8: Mispredictions over the compexity of the branch predictors

Complex predictors excel only when their extra history or non-linear modeling matches the workload’s branch structure. Here, per-branch local history delivered the best accuracy while the heavyweights gained little and, in TAGE’s case, lost a bit—likely because the benchmarks lack very long or irregular history patterns for it to exploit.

Local and TAGE mispredict roughly as often, but they mispredict under different conditions. Local history mainly struggles with branches that have long dependence chains. Each miss on such a branch flushes more in-flight work. L-TAGE often fails on hard branches that demand very long or unusual history patterns—exactly the ones whose mispredictions flush the deepest portion of the pipeline and stall the fetch unit the longest.

6 Conclusion

This study based on simulations shows that there is no one best branch predictor; the performance of a branch predictor depends a lot on the control flow characteristics of the workload. The TAGE predictor consistently outperformed the others by getting the highest overall IPC and the lowest misprediction rates. This was possible because it could capture a wide range of branch history patterns with its multiple tagged tables. In general, more complex predictors did better than simpler ones. However, the results for Tournament and LTAGE were mixed, showing that their hybrid strategies can sometimes make things worse for some workloads.

References

- [1] N. L. Binkert, B. M. Beckmann, G. Black, *et al.*, “The gem5 simulator,” *SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011. DOI: [10.1145/2024716.2024718](https://doi.org/10.1145/2024716.2024718). [Online]. Available: <https://doi.org/10.1145/2024716.2024718>.
- [2] J. Lowe-Power, A. M. Ahmad, A. Akram, *et al.*, “The gem5 simulator: Version 20.0+,” *CoRR*, vol. abs/2007.03152, 2020. arXiv: [2007.03152](https://arxiv.org/abs/2007.03152). [Online]. Available: <https://arxiv.org/abs/2007.03152>.
- [3] A. Hansson, N. Agarwal, A. Kolli, T. F. Wenisch, and A. N. Udipi, “Simulating DRAM controllers for future system architecture exploration,” in *2014 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2014, Monterey, CA, USA, March 23-25, 2014*, IEEE Computer Society, 2014, pp. 201–210. DOI: [10.1109/ISPASS.2014.6844484](https://doi.org/10.1109/ISPASS.2014.6844484). [Online]. Available: <https://doi.org/10.1109/ISPASS.2014.6844484>.